**Register Number:** 192425207

**Name:** Anand Paul. D

**CO5– AT2**

**MLA0201 – Fundamentals of Machine Learning**

**Q1. Monte Carlo Sampling for Probability Estimation**

**Pseudocode**

```
ALGORITHM MonteCarloProbability

INPUT:
   N = number of simulations

OUTPUT:
   Estimated probability of on-time delivery

BEGIN
   OnTimeCount ← 0

   FOR i ← 1 TO N DO

      Generate random traffic condition

      Simulate delivery based on traffic condition

      IF delivery is on time THEN
         OnTimeCount ← OnTimeCount + 1
      END IF

   END FOR
```

Probability ← OnTimeCount / N

DISPLAY "Estimated Probability =", Probability

END

**Explanation**

- Generate random traffic conditions for each trial.
- Simulate the delivery.
- Count the number of on-time deliveries.
- Divide on-time deliveries by total simulations.
- The result gives the estimated probability.

**Formula:**

$$P(\text{On-time}) = \frac{\text{Number of on-time deliveries}}{\text{Total simulations}}$$

**Q2. ε-Greedy Algorithm for K-Armed Bandit**

**Pseudocode**

ALGORITHM EpsilonGreedy

INPUT:

K = number of movies

N = number of user interactions

ε = exploration probability

OUTPUT:

Movie with highest expected reward

BEGIN

FOR i ← 1 TO K DO

```
    RewardSum[i] ← 0
    Count[i] ← 0
    AverageReward[i] ← 0
END FOR

FOR t ← 1 TO N DO

    Generate random number r between 0 and 1

    IF r < ε THEN
        Choose a random movie
    ELSE
        Choose movie with highest AverageReward
    END IF

    Observe user engagement reward

    Count[chosen_movie] ← Count[chosen_movie] + 1

    RewardSum[chosen_movie] ←
        RewardSum[chosen_movie] + reward

    AverageReward[chosen_movie] ←
        RewardSum[chosen_movie] / Count[chosen_movie]

END FOR

BestMovie ← movie with highest AverageReward
```

DISPLAY "Best Movie =", BestMovie

DISPLAY "Expected Reward =", AverageReward[BestMovie]

END

**Explanation**

- Exploration: With probability ε, choose a random movie.
- Exploitation: With probability $1 - \epsilon$, choose the movie with the highest estimated reward.
- After every interaction, update the movie's reward estimate.
- Finally, select the movie with the highest expected reward.

**Q3. Value Iteration for Robot Path Planning**

Pseudocode

ALGORITHM ValueIteration

INPUT:

States S

Actions A

Rewards R

Transition probabilities P

Discount factor γ

Threshold θ

OUTPUT:

Optimal value function V

Optimal policy π

BEGIN

```
FOR each state s in S DO
    IF s is terminal THEN
        V[s] ← 0
    ELSE
        V[s] ← 0
    END IF
END FOR

REPEAT

    Δ ← 0

    FOR each state s in S DO

        OldValue ← V[s]

        BestValue ← -∞

        FOR each action a in A DO

            ExpectedValue ← 0

            FOR each next state s' DO
                ExpectedValue ← ExpectedValue +
                P(s'|s,a) ×
                (R(s,a,s') + γ × V[s'])
            END FOR

            IF ExpectedValue > BestValue THEN
```

```
            BestValue ← ExpectedValue
        END IF

    END FOR

    V[s] ← BestValue

    Δ ← MAX(Δ, |OldValue - V[s]|)

  END FOR

UNTIL Δ < θ

FOR each state s in S DO

  π[s] ← action having maximum
        expected value

END FOR

DISPLAY "Optimal Value Function =", V
DISPLAY "Optimal Policy =", π

END
```

**Main Bellman Equation**

$$\boldsymbol{V(s) = \max_{a} \sum_{s'} P\,(s', \mid sa)[R(s, a, s') + \gamma V(s')]}$$

Explanation

- Initialize values of all states.

- Calculate the expected value of every possible action.
- Select the action with the maximum value.
- Repeat until the values become stable.
- Generate the optimal policy from the final values.

**Q4. Temporal Difference (TD) Learning for Game Score Prediction**

**Pseudocode**

**ALGORITHM TD_Learning**

**INPUT:**

**Number of episodes**

**Learning rate α**

**Discount factor γ**

**OUTPUT:**

**Learned state-value function V**

**BEGIN**

**FOR each state s DO**

**V[s] ← 0**

**END FOR**

**FOR episode ← 1 TO NumberOfEpisodes DO**

**Initialize current state S**

**WHILE S is not terminal DO**

**Take an action**

**Observe reward R**

**Observe next state S'**

**V[S] ← V[S] +**

**α × (R + γ × V[S'] - V[S])**

**S ← S'**

**END WHILE**

**END FOR**

**DISPLAY "Learned State Values =", V**

**END**

**TD Learning Formula**

$$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$$

**Explanation**

- Start with initial state-value estimates.
- Take an action and observe the reward.
- Observe the next state.
- Calculate the TD error.
- Update the current state's value.
- Continue until all episodes are completed.

**TD Error:**

$$\boldsymbol{\delta = R + \gamma V(S') - V(S)}$$

---

**Q5. Policy Iteration for Autonomous Drone Navigation**

**Pseudocode**

ALGORITHM PolicyIteration

INPUT:

States S

Actions A

Rewards R

Transition probabilities P

Discount factor γ

OUTPUT:

Optimal Policy π

BEGIN

Initialize a random policy π for every state

REPEAT

/* Policy Evaluation */

REPEAT

Δ ← 0

FOR each state s in S DO

OldValue ← V[s]

Choose action a ← π[s]

V[s] ← Σ P(s'|s,a) ×

(R(s,a,s') + γ × V[s'])

Δ ← MAX(Δ, |OldValue - V[s]|)

END FOR

UNTIL Δ < θ

/* Policy Improvement */

PolicyStable ← TRUE

FOR each state s in S DO

OldAction ← π[s]

Find action a that gives maximum

expected value

π[s] ← a

IF OldAction ≠ π[s] THEN

```
            PolicyStable ← FALSE
        END IF

    END FOR

UNTIL PolicyStable = TRUE

DISPLAY "Optimal Policy =", π
DISPLAY "State Values =", V

END
```

**Explanation**

Policy Iteration has two main steps:

1. Policy Evaluation
    - Calculate the value of the current policy.
    - Repeat until the values converge.
2. Policy Improvement
    - Check all possible actions.
    - Select the action with the highest expected value.
    - If the policy does not change, it is optimal.